

Technical Recap: Building a Local Agentic AI Architecture

Presenter: Jeffery Uhrick **Focus:** Phases 1 & 2 (Single-Agent ReAct Loops, Vector Memory, and Production Infrastructure)

1. Executive Summary

This document outlines the transition from standard, linear Large Language Model (LLM) API calls to a fully autonomous, local AI agent architecture. Over three distinct development phases, we architected a local FastAPI microservice that utilizes open-source models (Gemma-4-e4b), an autonomous reasoning loop, a dual-layer memory system (S3 + ChromaDB), and a scalable production codebase.

2. Prerequisites & Critical Infrastructure Notes

Before running the architecture, the following environmental configurations and libraries are required.

- **Required Python Libraries:** `fastapi`, `uvicorn`, `boto3`, `langchain`, `langchain-core`, `langchain-openai`, `langchain-classic`, `langchain-chroma`, `langchain-huggingface`, `sentence-transformers`.
- **LocalStack Authentication (Crucial):** LocalStack now requires an Auth Token (`LOCALSTACK_AUTH_TOKEN`) across all tiers, including the Community edition. You must register at localstack.cloud, retrieve your token, and add it to your `.env` file. This must be explicitly passed into the `localstack` service block in `docker-compose.yml`.
- **Docker Volume Persistence:** To prevent the AI from suffering "amnesia" when the container restarts, local directories (`./docs`, `./local_memory_backup`, `./chroma_data`) must be mapped to the `/app` working directories inside the Docker configuration.
- **Environment Variable Decoupling:** Never hardcode configuration data. `LLM_BASE_URL`, bucket names, and API keys must be loaded dynamically using the `.env` file to ensure the application remains platform-agnostic.

3. Day 1: The Autonomous ReAct Engine & Key-Value Storage

The Objective: Move away from standard chat completion interfaces and build an AI that can *think, reason, and execute code* autonomously.

Architectural Additions:

- **The ReAct Loop:** We implemented LangChain's ReAct (Reason + Act) architecture. Instead of immediately returning text to the user, the AI generates an internal monologue ("Thought"), decides which Python function to run ("Action"), reads the output ("Observation"), and loops until the task is complete.
- **Key-Value Memory (S3):** We integrated the AWS `boto3` SDK with a local instance of LocalStack to simulate an S3 bucket. We bound two tools to the AI: `save_to_long_term_memory` and `read_from_long_term_memory`.

The Takeaway (Why & When): Key-Value storage (S3) is required for exact file generation (e.g., saving a Python script or a generated configuration file). However, a critical limitation emerged: the AI had to guess the exact filename to retrieve a memory. This proved that strict, named databases are too fragile for human-like recall, necessitating a semantic approach.

4. Day 2: Semantic Memory & The Local RAG Pipeline

The Objective: Give the AI the ability to search for information based on *concept* and *meaning* rather than strict key-value pairs.

Architectural Additions:

- **The Vector Database:** We deployed **ChromaDB** as our semantic storage layer.
- **The Math Engine:** We integrated `sentence-transformers` via **Hugging Face**. This allowed our local Python container to mathematically convert human sentences into high-dimensional arrays of numbers (embeddings) without relying on external cloud APIs.
- **The RAG Pipeline (Retrieval-Augmented Generation):** Because AI models have a strict "context window" limit, we built an `ingest_local_file` tool. This pipeline uses a `RecursiveCharacterTextSplitter` to safely slice massive local documents into overlapping 1000-character chunks before embedding them into ChromaDB.

The Takeaway (Why & When): Semantic Memory (ChromaDB) is used for fuzzy facts, user preferences, and querying large, unstructured datasets. By utilizing RAG, the AI can mathematically search a massive database for a specific concept, retrieve the single most relevant paragraph, and inject it into its working memory before formulating a response.

5. Day 3: Enterprise Architecture & Production Refactor

The Objective: Prepare the codebase for scalable, multi-agent systems by deprecating the monolithic script.

Architectural Additions:

- **Domain Isolation:** We separated the logic into a standard Python microservice structure:
 - `/core`: Handles infrastructure, lifespans, and environment variables.
 - `/api`: Strictly handles web traffic and FastAPI routing.
 - `/tools`: Isolates the Boto3 (S3) and ChromaDB functions for reuse.
 - `/services`: Contains the ReAct loop and LLM initialization.
- **Sanitization:** We updated our ingestion tools to use `os.path.basename()` to automatically sanitize AI inputs, preventing directory traversal errors when the AI attempts to pass complex folder paths.

The Takeaway (Why & When): Separation of Concerns is mandatory for multi-agent systems. By extracting the configuration into `.env` files, our application is now completely model-agnostic. By isolating the tools into their own modules, future agents (e.g., a dedicated "Coder Agent" and a "Reviewer Agent") can import and utilize the exact same Python tools without duplicating logic or causing circular dependencies.

6. Current State & Next Steps

We have successfully deployed a robust, containerized, open-source AI microservice. The agent dynamically chooses between exact-match storage (S3) and semantic storage (ChromaDB) based entirely on the user's intent.

Phase 3 Roadmap: We will introduce **LangGraph**, transitioning from a single-agent loop into a cyclic, state-driven multi-agent architecture (e.g., a Supervisor node routing complex workloads between specialized worker agents).